

Payment Reminder Project Review

Current implementation and MSME readiness | 15 September 2026

1 Overall assessment

A useful proof of concept, with substantial work needed before production use. The project helps an accounts operator turn an Excel invoice list into reviewed payment-reminder emails. Its simple workflow is a good starting point for a small business. Today, it is a manually triggered batch assistant; dependable ongoing collections, payment tracking and support for multiple businesses still need to be built. A controlled pilot should follow the critical fixes below. Shared production deployment is premature.

2 How the project works

Setup and modes. The Python application has a Streamlit browser interface and a separate command-line agent. Local setup verifies a Groq AI key and Gmail App Password and collects the business name. When SPACE_ID is present, the interface switches to a Hugging Face demo using host-provided credentials. Dry run defaults to on; the demo hides the toggle even though its wording suggests emails will be sent.

Import and selection. The browser accepts .xlsx files and reads the first worksheet. Expected fields are Invoice No, Client Name, Email, Amount, Due Date and Status; Invoice Date and Notes also influence processing. Headers are normalized, values become strings, and dates are converted. The operator selects a 1-60 day overdue threshold, defaulting to seven. Filtering excludes paid, cancelled and cleared rows, then orders eligible invoices by days overdue. Missing due dates fall back to invoice dates.

Draft, review and send. Groq, configured with llama-3.3-70b-versatile, drafts one email per overdue invoice. The browser caches drafts and normally falls back to a fixed template on API or parsing errors. Messages use rupees, English and a fixed request to pay within three working days. Each send toggle starts on; the operator can edit subject/body or skip a recipient before clicking the batch Send button. Gmail SMTP sends each email sequentially. The screen shows results and offers a CSV reminder log.

The command-line path differs. An AI tool-calling loop chooses when to read, filter, request terminal approval, send and log; it stops after at most 40 model iterations. The browser directly orchestrates these steps and does not run this agent. Consequently, approval behavior, drafting and error handling have two implementations.

3 Code and data architecture

Component	Responsibility
app.py	Browser pages, setup, session state, AI drafting, review, sending and result display.
agent.py and tool_schemas.py	Command-line options, AI instructions, model loop, tool dispatcher and five tool definitions.
tools.py	Excel reader, overdue calculation, terminal approval, Gmail transport and CSV logging.
config.py	Environment settings for credentials, model, business, file paths and dry run.
setup_sample.py and project files	Six sample invoices; dependency declarations; minimal hosting metadata. Ignore rules cover only Python caches.

Where information lives. Uploaded workbooks pass through temporary disk files; current invoices, drafts and results live in server memory tied to each browser session. A local CSV holds timestamps, invoice/contact details, amounts and a 120-character body preview. There is no database, customer ledger or invoice-status write-back. The log is never checked before sending. Browser drafting sends selected invoice details and notes to Groq; the command-line loop sends the reader's entire invoice result to the model. Gmail receives the outgoing messages.

Engineering quality. Small, readable functions and shared I/O tools make the project understandable and adaptable. Human review, default dry run and a template fallback are useful foundations. However, there is no included test suite, automated build checks, dependency lock, scheduler, durable job queue or monitoring. The README is mostly hosting metadata. Its built-in Streamlit deployment setting is now deprecated by Hugging Face, which recommends Docker for Streamlit [1].

4 Main findings and business impact

Priority	Finding and consequence
Critical	Shared deployment lacks isolation. There is no application login or business-level access control. Sessions write credentials and dry-run settings into shared module globals, creating interference risk. The download exposes the same complete CSV log to each session. A shared host can therefore expose other customers' reminder records. Code: app.py 554-557, 622-630.
Critical	Approval is not enforced by the command-line backend. Its dispatcher accepts a send request without verifying prior approval or binding it to the reviewed recipient and text. A local fake-transport check confirmed this path. Untrusted invoice content also enters the model context. --no-dry-run enables sending while its banner can still say dry run. Code: agent.py 81-116, 135, 257-260.
High	Amounts and invoice eligibility are unreliable for messy data. Missing columns are accepted; malformed dates are silently dropped; blank/unknown statuses can qualify. An invoice date can become an unintended due date. The sample's INR 12,500 invoice with INR 2,000 received still retains INR 12,500 as its amount. No structured partial-payment balance exists. Code: tools.py 37-110; setup_sample.py 40-55.
High	Repeat runs can resend, and history cannot prove delivery. No duplicate-send protection or reminder cooldown exists. Dry runs are logged without a mode/status distinction. Logging failures are ignored by the browser, which can still report sent. SMTP success records submission, not customer delivery or payment. Code: app.py 565-572; tools.py 181-269.
High	Errors and setup can mislead the operator. Import failure and no overdue invoices both return an empty list, followed by an all-payments-on-track message. Valid JSON without a body can produce a blank draft. Fresh setup only saves credentials if .env already exists; Start over clears session credentials. .env and invoice/log files are not excluded by the supplied ignore rules. Code: app.py 145-173, 275-277, 331-334, 413-419, 644-646.

5 Prioritized improvements for a generalized MSME product

P0 - Make the existing workflow trustworthy. Validate required fields, dates, email addresses, amounts and unique invoice IDs, with row-level correction messages. Store invoices, outstanding balances, approvals and reminder attempts durably. Enforce approval against the exact recipient/message, prevent duplicate sends, separate dry-run records, and surface failures. Add secure business accounts, isolated credentials/data, backups and regression tests. Fix setup and deployment before onboarding a shared pilot.

P1 - Build the daily collections workflow. Support partial payments, credits, disputes, payment promises and follow-up pauses. Add configurable reminders before/on/after due dates, working hours, escalation and automatic stopping when payment is confirmed. Offer customer-wise statements, aging buckets (1-30, 31-60, 61-90, 90+ days), owner assignments and collections reports. Let each business configure branding, currency, language, terms and tone. Prioritize mobile-friendly email and WhatsApp workflows with contact preferences and opt-out handling.

P2 - Connect collection to payment. Add flexible Excel/CSV column mapping, then accounting connectors selected with pilot users; TallyPrime and Zoho Books are documented integration candidates [2,3]. Add payment links/UPI options and verified payment-event handling to match receipts to invoices [4]. Later, add a customer portal, installments and cash-flow forecasts grounded in payment history. These are proposed additions, not existing capabilities.

Recommended direction. Retain the useful interface and import helpers, but centralize business rules in one service used by both entry points. Put a database and background worker behind it for durable scheduled work. Keep balances, eligibility and approvals deterministic; use AI for optional wording. Generalization should come from configurable business rules and connectors. Measure the pilot by correct balances, zero unintended duplicate reminders, delivery failures and recorded collections. Plan for provider quotas and operating costs; Groq applies rate limits [5].

Review evidence. All 10 supplied project files were reviewed, including six Python files totaling 1,424 lines. Nineteen isolated checks covered syntax, the sample workflow and failure cases; the sample produced the expected four overdue invoices. External AI/email services were replaced with fakes. Live credentials, browser behavior, delivery, concurrency and deployment were not validated. Application source was not changed.

Official references: [1] Hugging Face Streamlit hosting · [2] TallyPrime integration · [3] Zoho Books API · [4] Razorpay payment links and payment events · [5] Groq rate limits